

Autonomous First Responder Fire & Rescue Robot User Manual & Technical Documentation

CSE 396 – Computer Engineering Project
Gebze Technical University – Group 7

Nuri Ziya Kırtepe (210104004027)
Ömer Nacar (210104004814)
Tuana Melisa Aksoy (230104004903)
Evrin Doğa Solmaz (230104004042)
Uğur Anıl Güney (210104004011)
Dicle Çoban (220104004088)
Fatma Öztürk (230104004152)
Gabil Rahimli (230104004902)

June 2026

Project Repository: <https://github.com/nziyak/GTU-CSE396-25.26-Group7>
Website: <https://nziyak.github.io/GTU-CSE396-25.26-Group7/>

Abstract

This document is the official User Manual and Technical Specification for the *Autonomous First Responder Fire & Rescue Robot*, designed and built by Group 7 for the CSE 396 Computer Engineering Project course. The system is an Edge-AI driven autonomous robot that enters disaster zones, detects victims using Tri-Modal Sensor Fusion (Visual, Acoustic, Environmental), classifies victim severity using a two-stage neural pipeline (YOLOv8 + CNN), and provides real-time telemetry to a Unity Digital Twin operator interface. The hardware layer uses an Arduino Mega 2560 (migrated from STM32F103C6 due to SWD pin damage) for motor control and sensor acquisition, while a Raspberry Pi 5 handles all AI inference, communication, and decision-making. This document provides complete hardware pinouts, communication protocol definitions, installation procedures, operational guides, and troubleshooting references.

Contents

1	Introduction	3
1.1	Project Scope	3
1.2	Hardware Change Notice	3
2	Hardware Architecture	3
2.1	Processing Nodes	3
2.2	Sensor Pin Assignments (Arduino Mega 2560)	3
2.3	Power Architecture	3
3	Communication Protocol	4
3.1	Serial Link Configuration	4
3.2	Command Packets (Pi → Mega)	4
3.3	Telemetry Packets (Mega → Pi)	5
4	Software Architecture	5
4.1	Decision Engine — AutonomyController	5
4.2	Two-Stage Vision Pipeline	5
4.3	Key Software Modules	5
5	Installation and Setup	6
5.1	Arduino Mega Firmware Upload	6
5.2	Raspberry Pi 5 Environment	6
5.3	Hardware Wiring Checklist	6
6	Operational Procedures	7
6.1	Startup Checklist	7
6.2	Starting the System	7
6.3	Unity Digital Twin Connection	7
7	Troubleshooting	7
8	Design Notes and Known Constraints	8

1 Introduction

1.1 Project Scope

In urban fire and earthquake scenarios, first responders face life-threatening conditions when searching for trapped victims in collapsed or smoke-filled structures. The *Autonomous First Responder Fire & Rescue Robot* is designed to enter hazardous zones before human firefighters, autonomously mapping the environment and locating victims using a tri-modal sensing approach:

1. **Visual Detection:** YOLOv8 human detection + custom CNN severity classification (TRAPPED / LYING / STANDING)
2. **Acoustic Localization:** Dual-microphone bearing estimation for distress call homing
3. **Environmental Monitoring:** Temperature, humidity, smoke/gas concentration tracking

All processing runs locally on the Raspberry Pi 5 with zero cloud dependency.

1.2 Hardware Change Notice

IMPORTANT: *The original hardware design used an STM32F103C6 (Blue Pill) microcontroller. Due to electrical damage to the SWD (Serial Wire Debug) programming pins, the STM32 became unprogrammable. The system was migrated to an **Arduino Mega 2560** before the project demo. All sensor connections and motor control logic remain identical; only the MCU, pin assignments, and firmware framework changed (STM32 HAL → Arduino Wiring).*

2 Hardware Architecture

2.1 Processing Nodes

Table 1: System Processing Nodes

Node	Hardware	Role
Edge AI Computer	Raspberry Pi 5 (8GB RAM) + Active Cooler	AI inference, decision engine, dashboard
Sensor Controller	Arduino Mega 2560 (ATmega2560, 16MHz)	Motor control, sensor acquisition
Previous MCU	STM32F103C6 (Blue Pill)	Replaced – SWD pin damage

2.2 Sensor Pin Assignments (Arduino Mega 2560)

2.3 Power Architecture

- **Raspberry Pi 5:** Dedicated USB-C PD powerbank
- **Motor & Sensor Power:** A separate powerbank feeds a decoy trigger cable to provide a stable 12V output. This 12V rail powers the L298N motor driver, sensors, and the Arduino Mega.
- **Ground Policy:** All GND rails (12V decoy, L298N, Mega, Pi) must be tied together (star topology) to ensure stable signal references.

Table 2: Sensor Pin Mapping

Sensor	Pin(s)	VCC	Type
HC-SR04 Front Distance	TRIG: 9, ECHO: 10	5V	Digital
HC-SR04 Back Distance	TRIG: 7, ECHO: 8	5V	Digital
MAX4466 Microphone 1	A0	3.3V	Analog (ADC)
MAX4466 Microphone 2	A1	3.3V	Analog (ADC)
MQ-2 Smoke Sensor	A2	5V	Analog (ADC)
DHT11 Temp/Humidity	Pin 4	5V	Digital (1-Wire)
MPU6050 IMU	SDA: 20, SCL: 21	3.3V	I2C

Table 3: Motor Driver (L298N) Pin Mapping

L298N Pin	Mega Pin	Function
ENA	Pin 5 (PWM)	Left motor speed control
IN1	Pin 22	Left motor direction 1
IN2	Pin 23	Left motor direction 2
IN3	Pin 24	Right motor direction 1
IN4	Pin 25	Right motor direction 2
ENB	Pin 6 (PWM)	Right motor speed control

IMPORTANT: The MPU6050 is NOT 5V tolerant. Its VCC must be connected to the Mega's 3.3V pin, not 5V.

3 Communication Protocol

3.1 Serial Link Configuration

Communication between the Raspberry Pi 5 and Arduino Mega operates over USB Serial:

- **Port:** /dev/ttyUSB0
- **Baud Rate:** 115200 bps
- **Data Format:** 8N1 (8 data bits, no parity, 1 stop bit)

3.2 Command Packets (Pi → Mega)

ASCII commands terminated by \n. Each command triggers 2-second timed motor execution.

Table 4: Command Set

Command	Behavior
FORWARD\n	Move forward for 2 seconds
BACKWARD\n	Move backward for 2 seconds
LEFT\n	Tank-turn left for 2 seconds
RIGHT\n	Tank-turn right for 2 seconds
STOP\n	Immediate stop

3.3 Telemetry Packets (Mega → Pi)

Every 300ms, the Mega transmits a JSON object:

```
{"dist_front":45,"dist_back":80,"mic1":340,"mic2":335,
  "smoke":420,"yaw":1.2,"pitch":-0.5,"roll":0.3,
  "temp":24.5,"hum":45.0,"connected":true}
```

Table 5: Telemetry JSON Fields

Field	Type	Description
dist_front / dist_back	int	Ultrasonic distance (cm)
mic1 / mic2	int	Microphone analog values (0–1023)
smoke	int	MQ-2 smoke sensor raw value (0–1023)
yaw / pitch / roll	float	MPU6050 orientation (degrees)
temp / hum	float	DHT11 temperature (°C) / humidity (%)
connected	bool	Mega connection alive flag

4 Software Architecture

4.1 Decision Engine — AutonomyController

The central decision module (`autonomy_controller.py`) implements a priority-based decision tree that fuses multi-modal sensor data:

1. **Priority 1 — Obstacle Avoidance:** If `dist_front` ≤ 18 cm, the robot immediately executes an alternating LEFT/RIGHT turn to avoid collision.
2. **Priority 2 — Victim Approach:** If YOLOv8 + CNN detects a TRAPPED or LYING victim, the controller steers the robot toward the target by comparing bounding box center against frame center (± 70 px tolerance). The robot stops when estimated distance ≤ 60 cm.
3. **Priority 3 — Autonomous Patrol:** When no obstacle or victim is detected, the robot moves FORWARD in patrol mode.

The decision cycle runs at 4 Hz (250ms intervals). Each decision generates a motor command sent to the Mega via UART.

4.2 Two-Stage Vision Pipeline

1. **Stage 1 — YOLOv8n:** Detects humans in Pi Camera V3 frames (640×480). Selects the best candidate by confidence $\times$ bbox area.
2. **Stage 2 — Custom CNN (ONNX):** Crops the detected person, resizes to 160×160 , and classifies severity: TRAPPED, LYING, or STANDING. Uses a 2-second sliding window for stable predictions.

4.3 Key Software Modules

- `pi_main.py` — System integration and main loop
- `autonomy_controller.py` — Decision engine

- `ai_vision.py` — Two-stage vision pipeline
- `uart_reader.py` — Serial telemetry reader/writer
- `comms_dashboard.py` — Flask/SocketIO web dashboard
- `stt_engine.py` — Offline Vosk speech-to-text
- `acoustic_homing.py` — Acoustic source localization

5 Installation and Setup

5.1 Arduino Mega Firmware Upload

1. Install Arduino IDE 2.x from <https://www.arduino.cc>
2. Install required libraries via *Sketch* → *Include Library* → *Manage Libraries*:
 - MPU6050_light by rfetick
 - DHT sensor library by Adafruit
 - Adafruit Unified Sensor
3. Select board: *Tools* → *Board* → *Arduino Mega or Mega 2560*
4. Select processor: *ATmega2560 (Mega 2560)*
5. Select port and click **Upload**
6. After upload, disconnect from PC and connect to Pi 5 USB port

5.2 Raspberry Pi 5 Environment

```
# System update
```

```
sudo apt update && sudo apt upgrade -y
```

```
# Python dependencies
```

```
pip install flask flask-socketio pyserial ultralytics
```

```
pip install opencv-python-headless onnxruntime picamera2 vosk
```

```
# Clone repository
```

```
git clone <repo-url> && cd robot_project1
```

5.3 Hardware Wiring Checklist

1. Connect L298N: ENA→Pin5, IN1→22, IN2→23, IN3→24, IN4→25, ENB→Pin6
2. Connect HC-SR04 front: TRIG→9, ECHO→10. Back: TRIG→7, ECHO→8
3. Connect MAX4466 microphones to A0, A1 (VCC→3.3V)
4. Connect MQ-2 to A2 (VCC→5V)
5. Connect DHT11 to Pin 4 (VCC→5V)
6. Connect MPU6050: SDA→20, SCL→21 (VCC→**3.3V only!**)
7. Tie all GND rails together (common ground)

6 Operational Procedures

6.1 Startup Checklist

1. Verify motor battery is 12V (9V is insufficient for L298N)
2. Confirm all ground connections are tied
3. Ensure Mega USB cable is connected to Pi 5
4. Verify Pi Camera V3 ribbon cable is seated in CSI port
5. Confirm `yolov8n.pt` and `severity_model.onnx` exist in project directory

6.2 Starting the System

```
# SSH into Pi 5
ssh pi@firebot.local
```

```
# Navigate to project
cd robot_project1
```

```
# Launch
python3 pi_main.py
```

Expected output:

```
[INFO] pi_main: Vision init successful
[WebDashboard] Starting server on 0.0.0.0:5001 ...
[INFO] pi_main: STM front=45 back=80 mic1=340 ...
```

6.3 Unity Digital Twin Connection

1. Open Unity project and enter Play mode
2. Set WebSocket URL to `ws://<Pi-IP>:5001`
3. HUD should display live telemetry within 2 seconds
4. Use operator controls for manual commands

7 Troubleshooting

- **Motors don't move:** Check battery voltage (use 12V). Verify ENA/ENB PWM pins. Check common ground.
- **Serial port not found:** Run `ls /dev/ttyUSB*`. Add user to dialout group.
- **MPU6050 reads zeros:** Verify SDA/SCL not swapped. Confirm VCC=3.3V.
- **Vision OOM crash:** STT pauses vision automatically. Reduce `exploration_fps` if needed.
- **DHT11 returns NaN:** Normal during warm-up. Firmware handles gracefully.
- **Bad UART JSON:** Motor noise causes occasional corruption. System retries automatically.
- **Ultrasonic unreliable:** MPU I2C conflicts with `pulseIn()`. Firmware sequences reads correctly.

8 Design Notes and Known Constraints

1. **2-Second Timed Commands:** Safety feature — if communication is lost, robot auto-stops after 2 seconds. The `AutonomyController` re-issues commands at 250ms to maintain motion.
2. **Thermal Management:** Pi 5 Active Cooler is essential. Without it, YOLO inference causes thermal throttling and FPS drops below 2.
3. **AVR Float Limitation:** Arduino AVR architecture does not support `snprintf %f`. Float values are output via `Serial.print(value, 1)`.
4. **Vision-STT Mutex:** Vision pipeline pauses during STT processing to prevent OOM on Pi 5's limited RAM.
5. **DHT11 vs DHT22:** The sensor was confirmed as DHT11 (not DHT22 as originally planned). Code uses `DHT11` type constant.